Friedrich-Alexander-Universität
Faculty of Engineering

Optimization of Photovoltaic and Battery Storage Systems

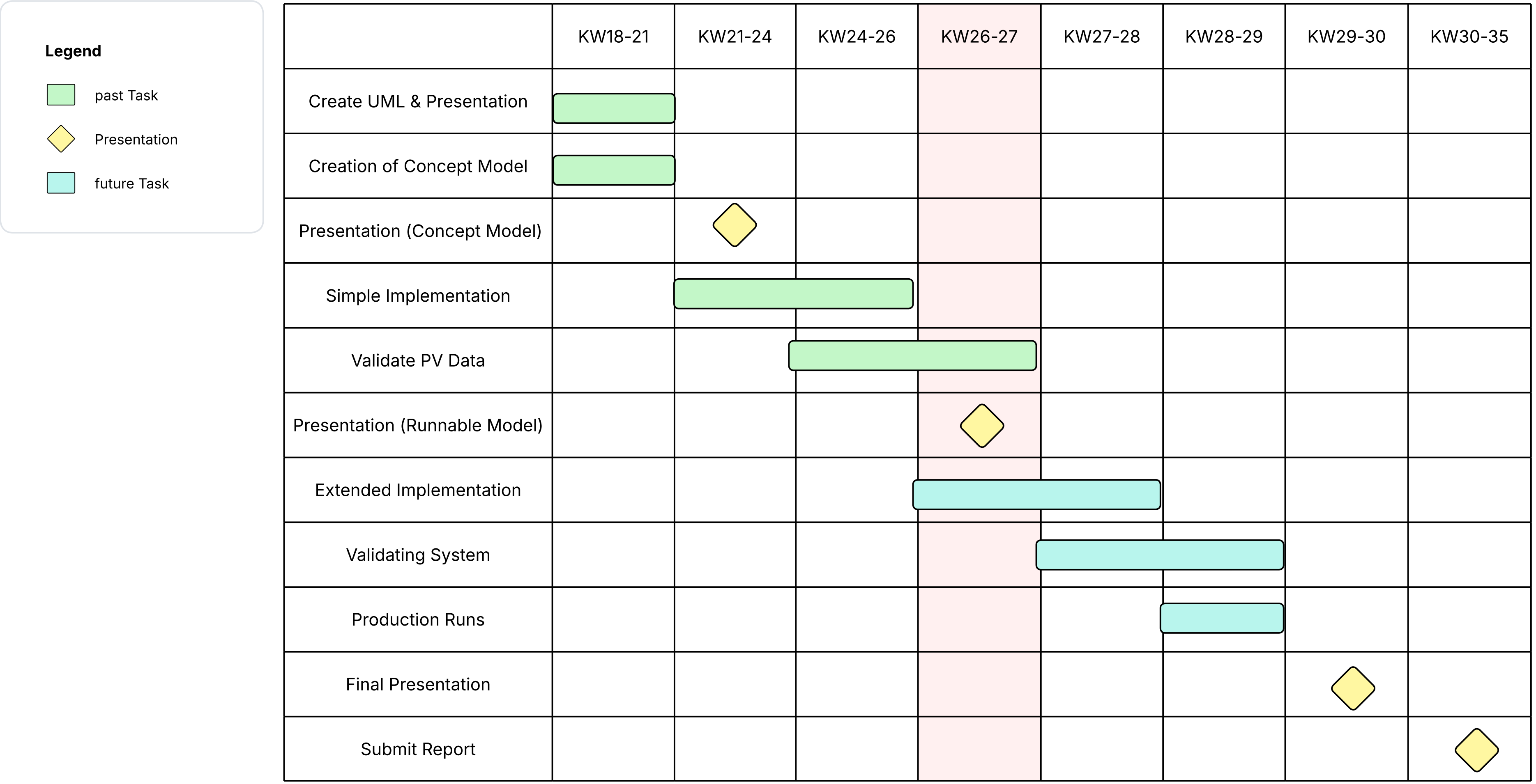
Robert Leichtl, Florian Merlau, Jakob Haberstock, Christian Besold, Nandini Joshi

Friedrich-Alexander-Universität Erlangen-Nürnberg, Computer Science 7

July 1, 2025

Contents

Project-Gantt



Contents

Initial Approach and Testing

- **Initial Approach:**
  - Utilization of AnyLogic's **internal HSQL database**.
  - Direct data import from CSV files **via Java**.
  - Data retrieval using a **Java API** for handover to other classes in AnyLogic.

Challenges During Implementation

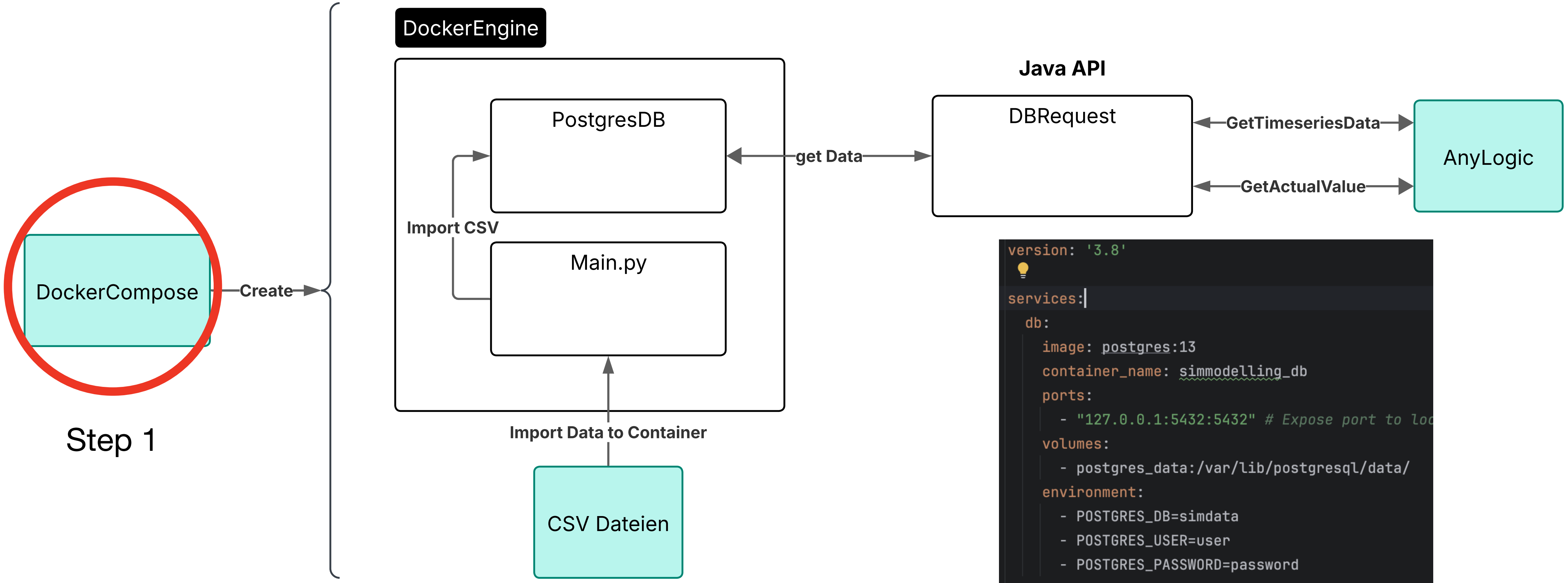
- **Problem with AnyLogic's Internal Database:**
  - **Faulty CSV Imports:** During testing of the implementation, errors occurred when importing large CSV files via Java into the HSQL database.

Solution: External PostgreSQL Database

Chosen Solution: Migration to an external PostgreSQL database, managed with Docker.

- **Data Import:**
  - CSV files are initially **imported into PostgreSQL using Python**.
  - **Advantage of Python:** Ideal for efficient data processing and import into SQL databases.
- **Data Retrieval:**
  - Data is still **retrieved from PostgreSQL via a Java API** and provided to AnyLogic classes.

Database

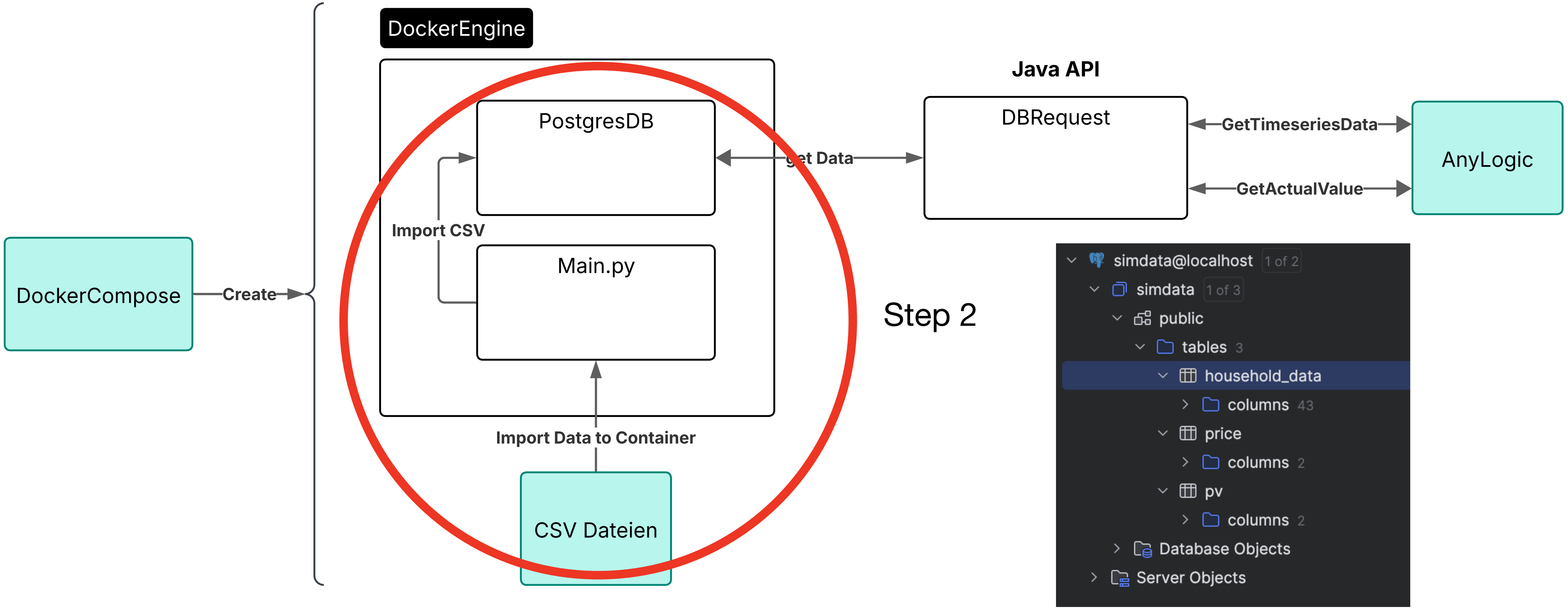



```yaml
version: '3.8'

services:
  db:
    image: postgres:13
    container_name: simmodelling_db
    ports:
      - "127.0.0.1:5432:5432" # Expose port to loc
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      - POSTGRES_DB=simdata
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
```

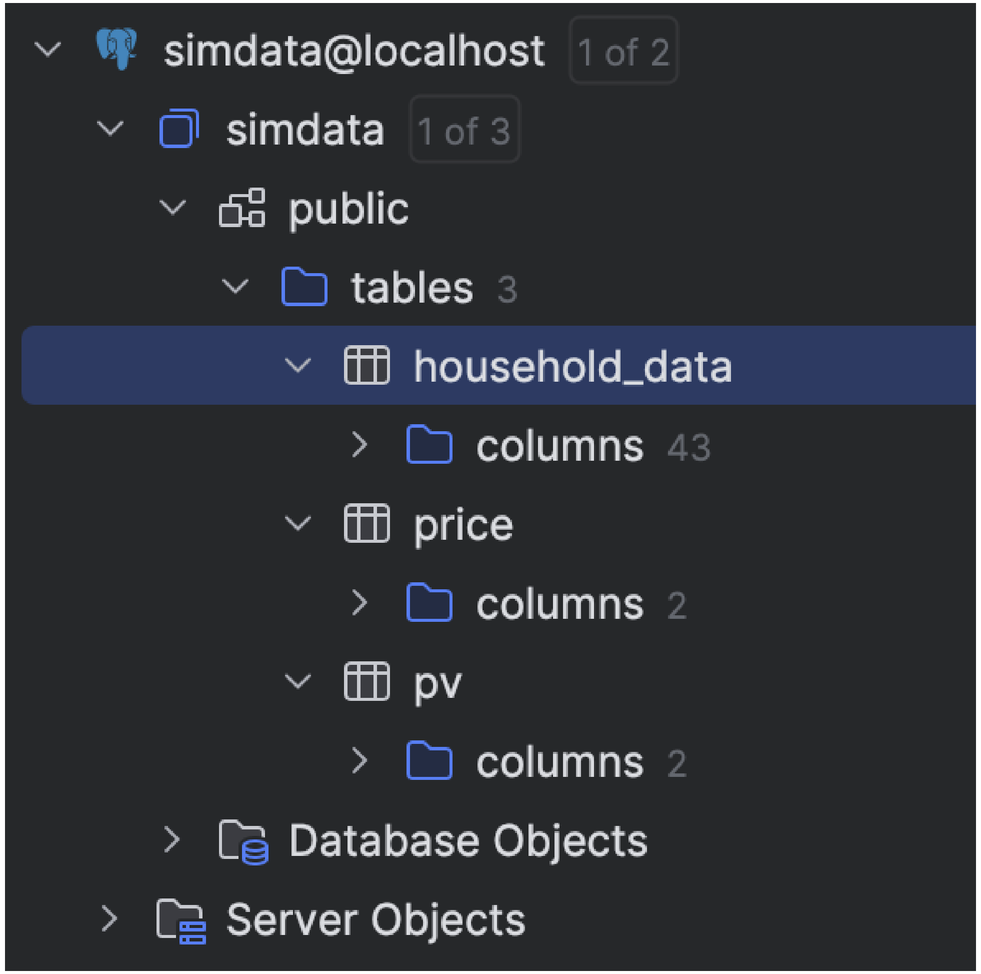
Database

```yaml
version: '3.8'

services:
  db:
    image: postgres:13
    container_name: simmodelling_db
    ports:
      - "127.0.0.1:5432:5432" # Expose port to localhost for security
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      - POSTGRES_DB=simdata
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password

  importer:
```

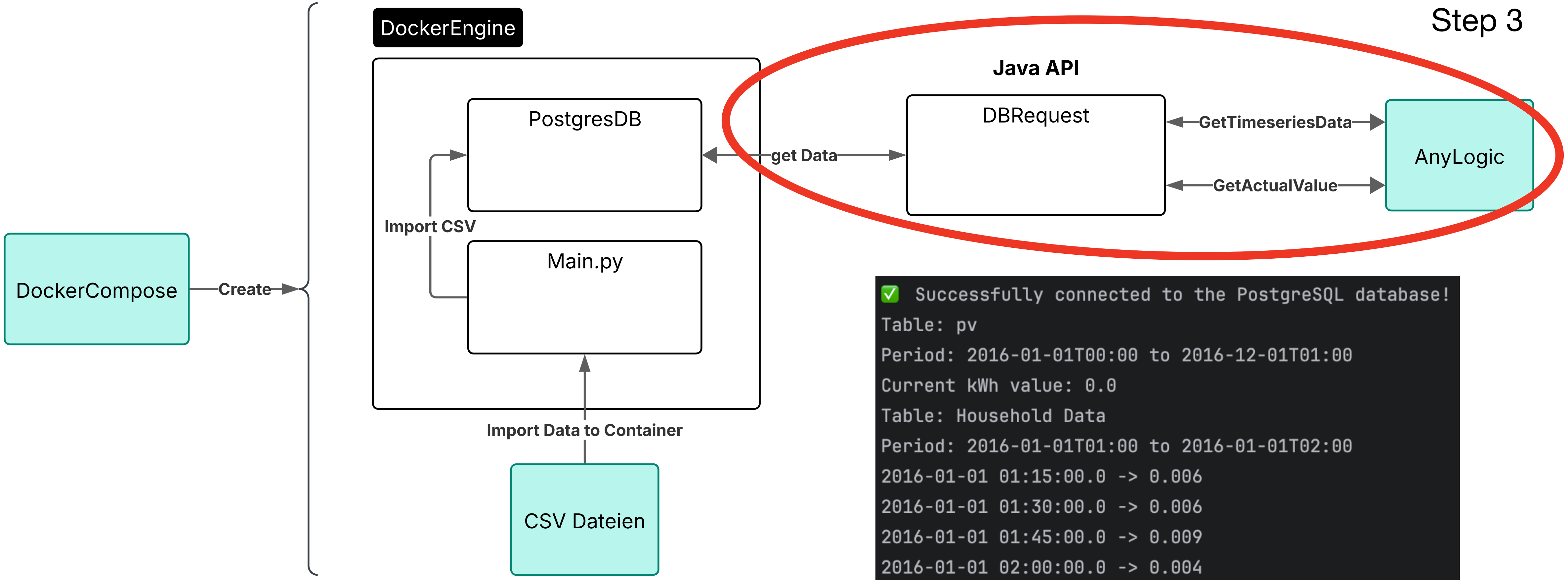
Database


DockerEngine
Java API
PostgresDB
DBRequest
GetTimeseriesData
AnyLogic
Get Data
GetActualValue
Import CSV
Main.py
DockerCompose
Create
Step 2
Import Data to Container
CSV Dateien
simdata@localhost 1 of 2
simdata 1 of 3
public
tables 3
household_data
columns 43
price
columns 2
pv
columns 2
Database Objects
Server Objects

Database


simdata@localhost 1 of 2
simdata 1 of 3
public
tables 3
household_data
columns 43
price
columns 2
pv
columns 2
Database Objects
Server Objects

Database



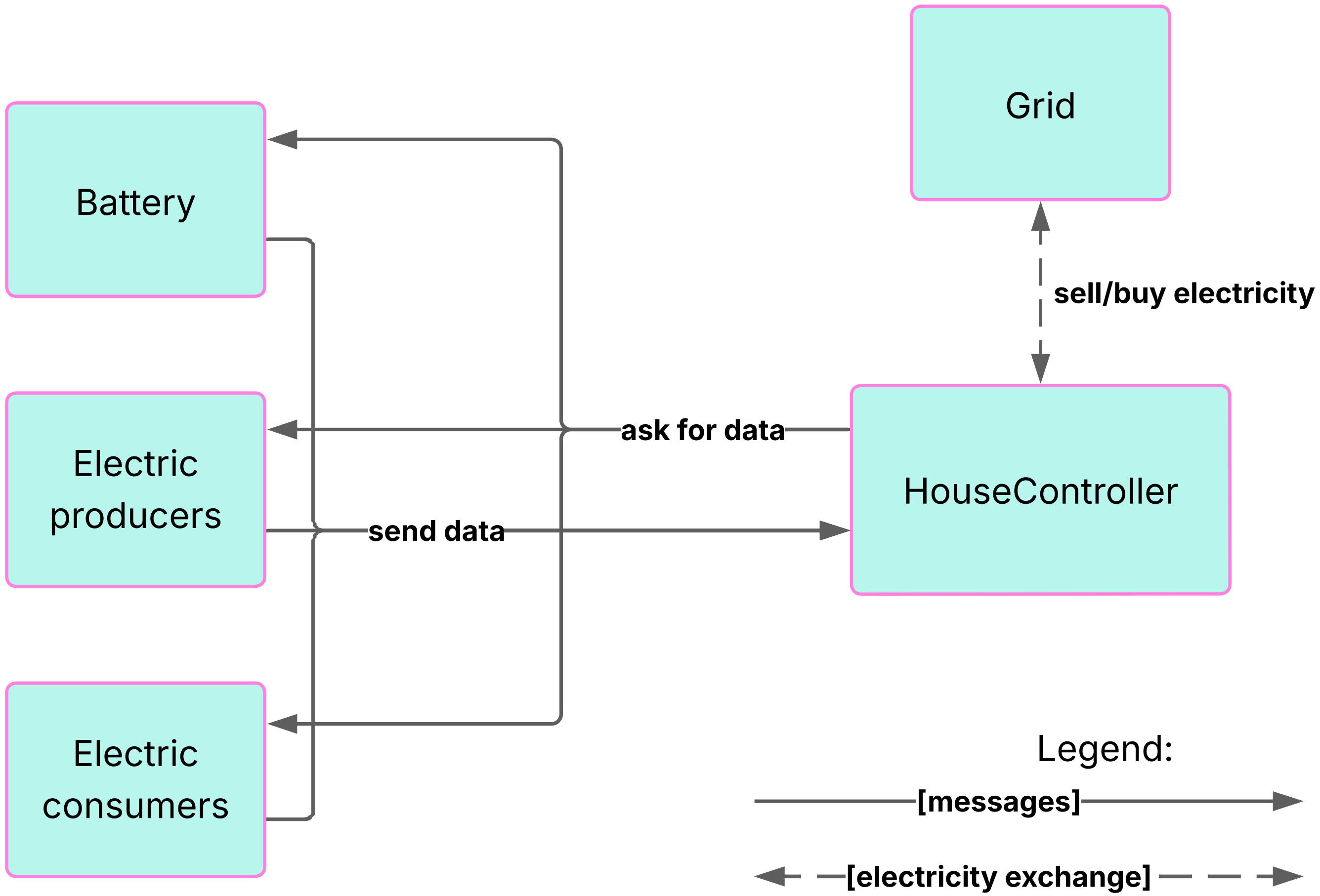

```
✅ Successfully connected to the PostgreSQL database!
Table: pv
Period: 2016-01-01T00:00 to 2016-12-01T01:00
Current kWh value: 0.0
Table: Household Data
Period: 2016-01-01T01:00 to 2016-01-01T02:00
2016-01-01 01:15:00.0 -> 0.006
2016-01-01 01:30:00.0 -> 0.006
2016-01-01 01:45:00.0 -> 0.009
2016-01-01 02:00:00.0 -> 0.004
```

Database

```
✅ Successfully connected to the PostgreSQL database!
Table: pv
Period: 2016-01-01T00:00 to 2016-12-01T01:00
Current kWh value: 0.0
Table: Household Data
Period: 2016-01-01T01:00 to 2016-01-01T02:00
2016-01-01 01:15:00.0 -> 0.006
2016-01-01 01:30:00.0 -> 0.006
2016-01-01 01:45:00.0 -> 0.009
2016-01-01 02:00:00.0 -> 0.004
```

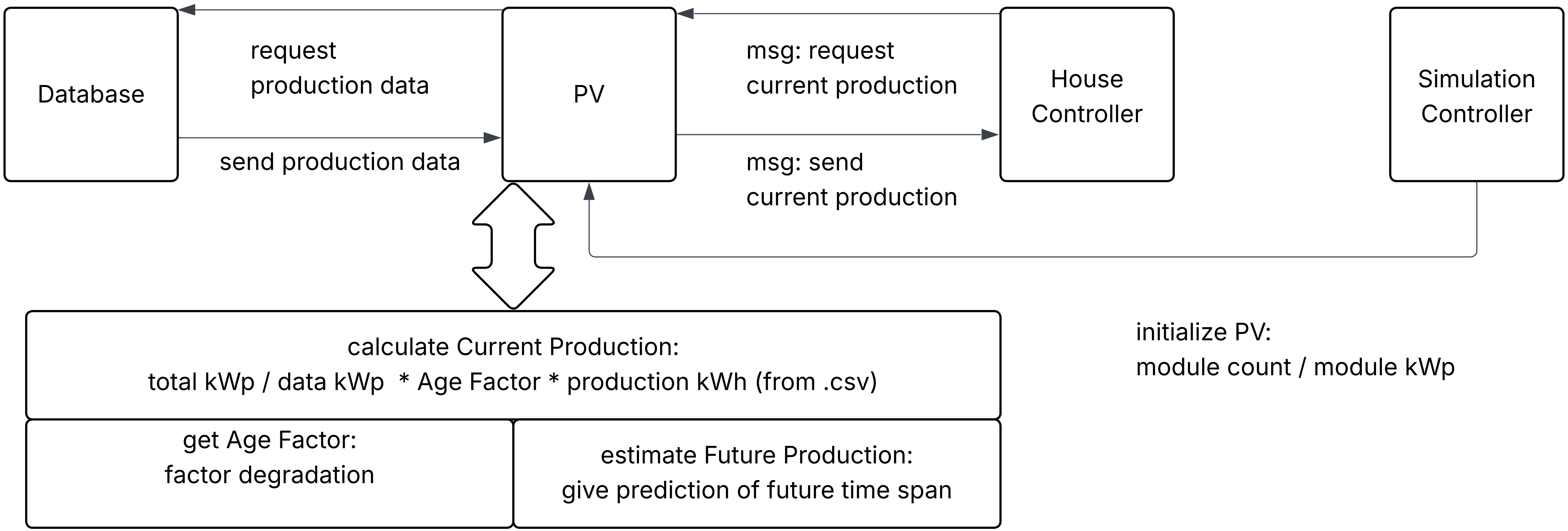
Contents

House Controller



Contents

PV


Database
request
production data
send production data
PV
msg: request
current production
msg: send
current production
House
Controller
Simulation
Controller
calculate Current Production:
total kWp / data kWp * Age Factor * production kWh (from .csv)
get Age Factor:
factor degradation
estimate Future Production:
give prediction of future time span
initialize PV:
module count / module kWp

PV.csv and price.csv

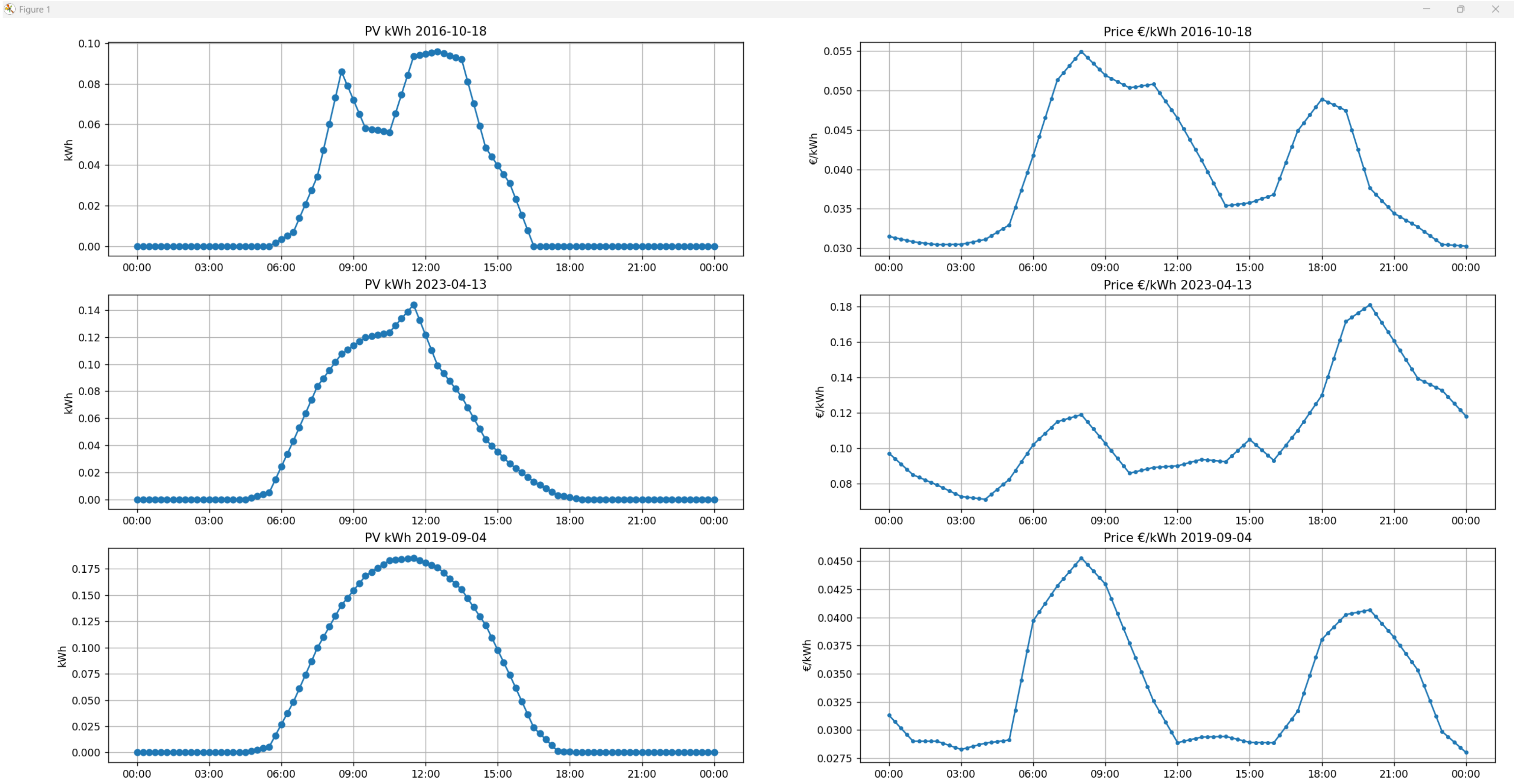
- Price source: Bundesnetzagentur[1]
- PV source: PVGIS[2]
- Reasons for choosing this PV source:
  Known kWp / free choice of location / few factors needed to calculate production
- Validation Problem: PVGIS provides simulated production data
  $\rightarrow$ Result must be validated before use
- Both datasets use 1 hour intervals
  $\rightarrow$ used linear interpolation to get 15 min intervals

Time	**Production (kWh)**
2005-01-01 14:45:00	0.0146
2005-01-01 15:00:00	0.0097
2005-01-01 15:15:00	0.0049
2005-01-01 15:30:00	0.0000

Time	**Price (€/kWh)**
2016-01-01 03:00:00	0.01681
2016-01-01 03:15:00	0.01696
2016-01-01 03:30:00	0.01711
2016-01-01 03:45:00	0.01726

[1] https://www.smard.de/home/downloadcenter/download-marktdaten/
[2] https://re.jrc.ec.europa.eu/pvg_tools/en/

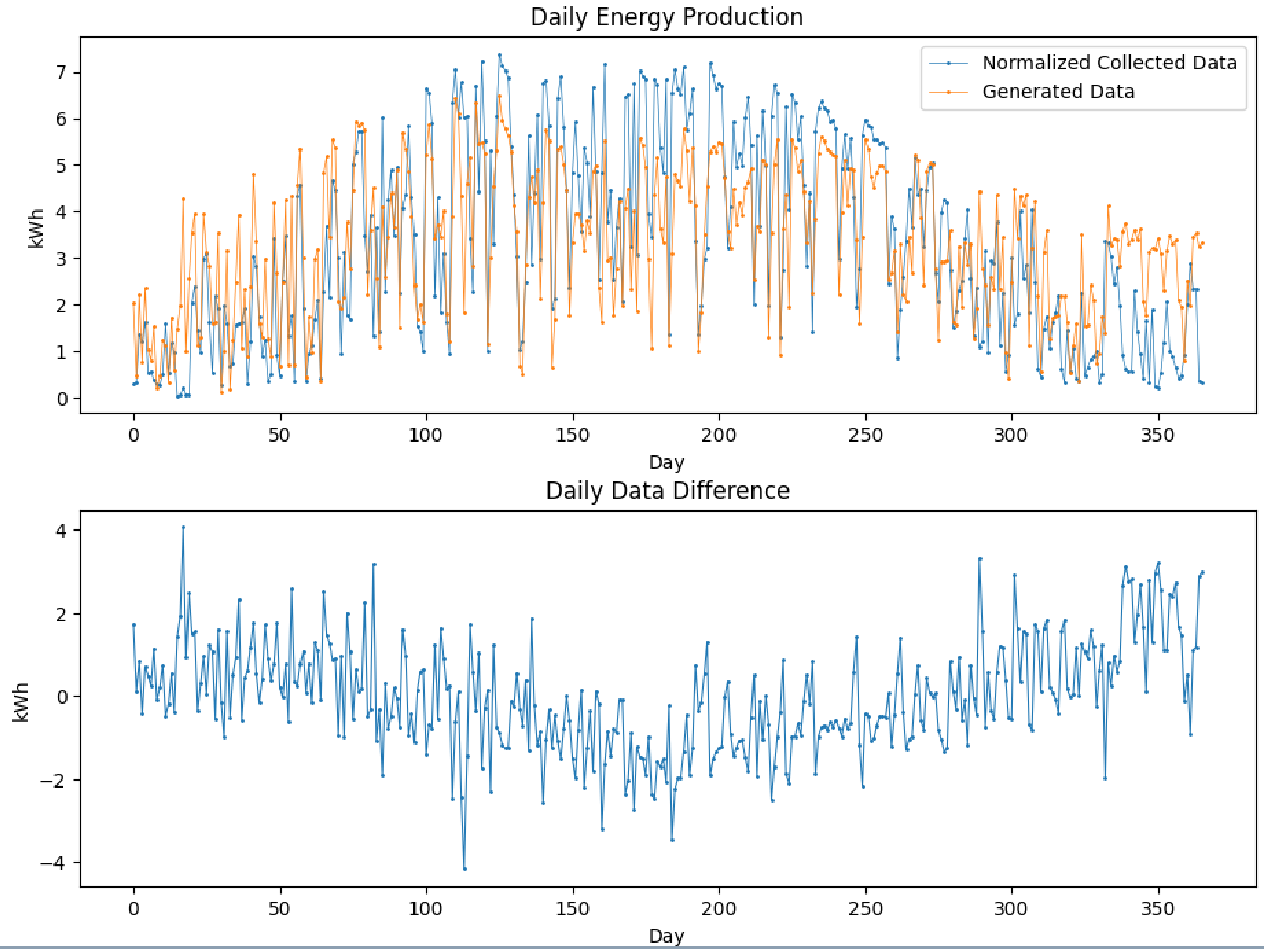
PV.csv and price.csv



PV Model Evaluation

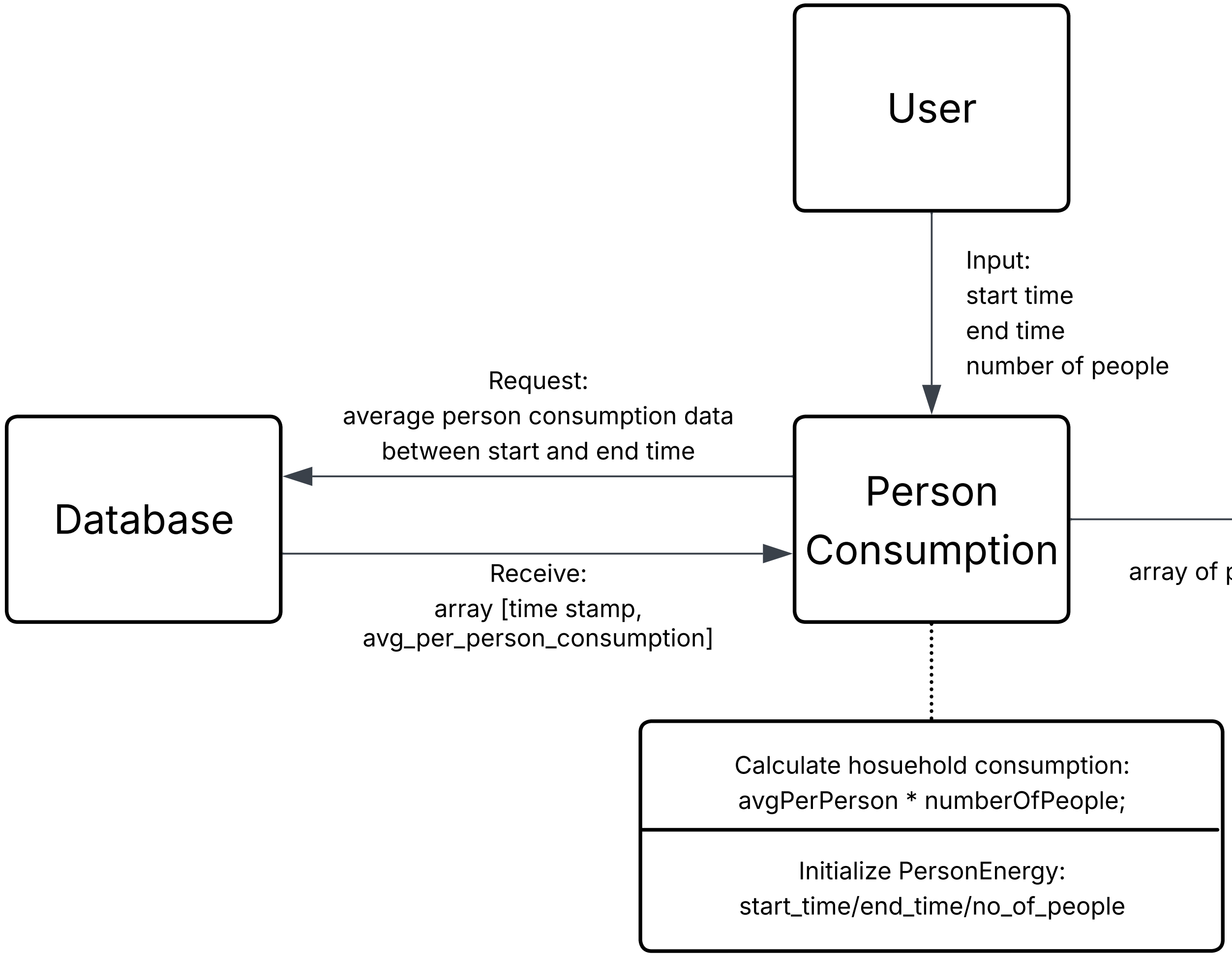
Model Performance

- **MAE:** 0.0153
- **RMSE:** 0.0010
- **R^2:** 0.7143
- **Pearson r:** 0.8470

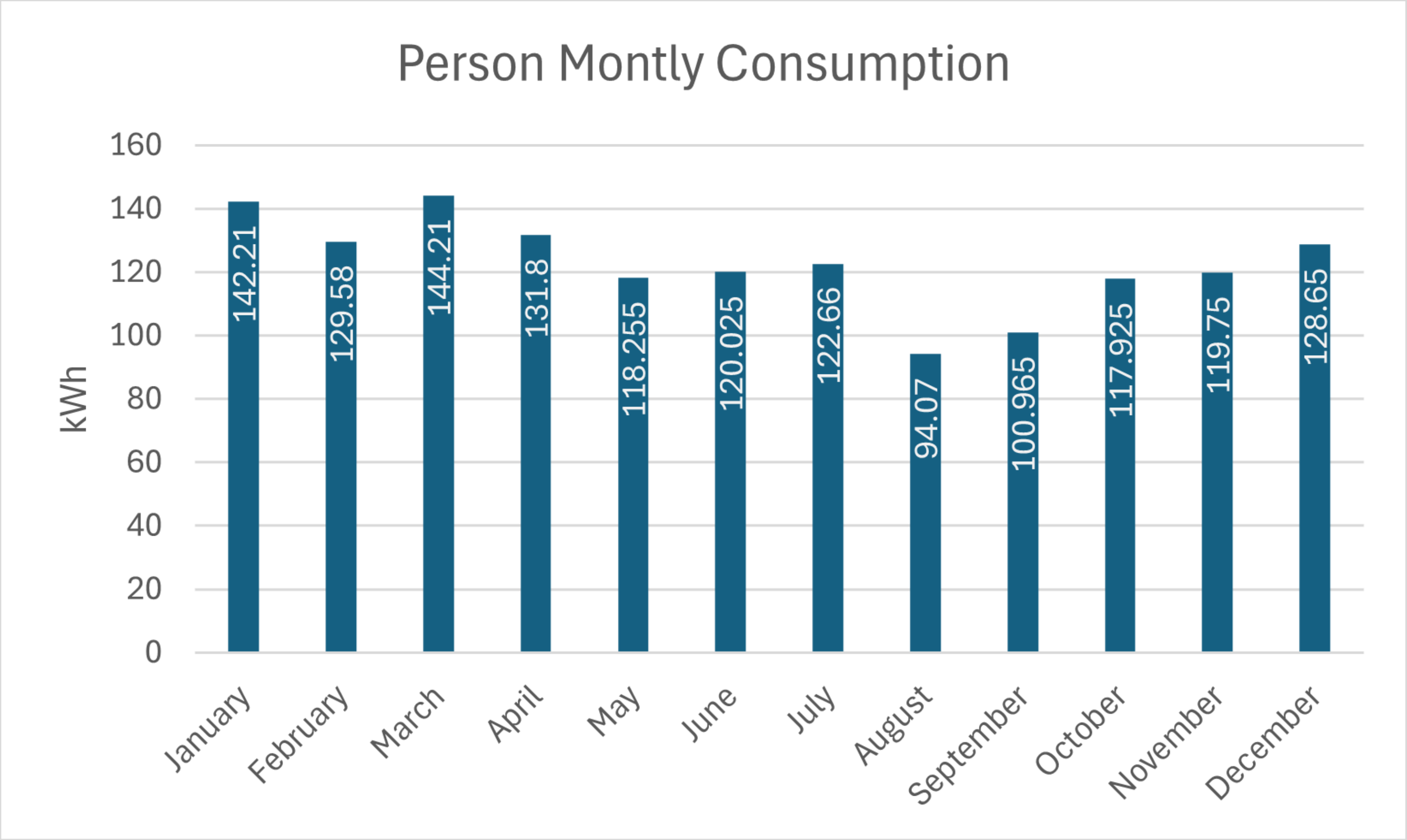


Contents

Person Consumption



Monthly Consumption



Technical Summary

Highlights:

- Accepts user defined inputs
- Requests and retrieves from the PostgreSQL database
- Calculates per-person consumption:

$$\text{avg\_per\_person\_consumption} \times \text{number\_of\_people}$$

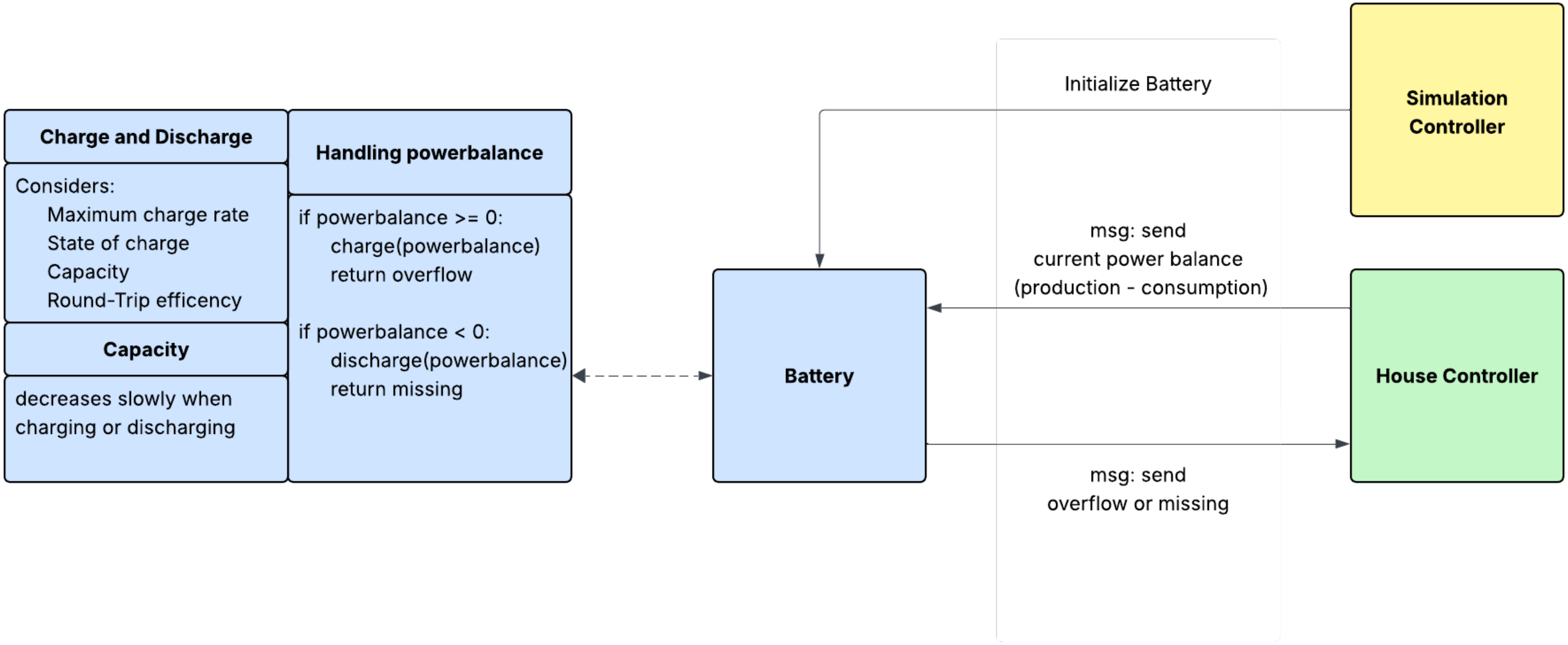
- Data Source: Kaggle - Household Load and Solar Generation[3]

utc_timestamp	**average_per_person_consumption (kWh)**
2016-01-01 00:00:00Z	0.000
2016-01-01 00:15:00Z	0.039
2016-01-01 00:30:00Z	0.029
2016-01-01 00:45:00Z	0.029
2016-01-01 01:00:00Z	0.046

[3] https://www.kaggle.com/datasets/mexwell/household-load-and-solar-generation

Contents

Battery


Charge and Discharge
Considers:
Maximum charge rate
State of charge
Capacity
Round-Trip efficency
Capacity
decreases slowly when charging or discharging
Handling powerbalance
if powerbalance >= 0:
charge(powerbalance)
return overflow
if powerbalance < 0:
discharge(powerbalance)
return missing
Battery
Initialize Battery
Simulation Controller
msg: send current power balance (production - consumption)
House Controller
msg: send overflow or missing

Contents

Live Demo

Live Demo

Thank You for Your Attention!

We are happy to answer your questions.
Let’s discuss and improve together!

Presented by:

Robert Leichtl, Florian Merlau, Jakob Haberstock, Christian Besold, Nandini Joshi